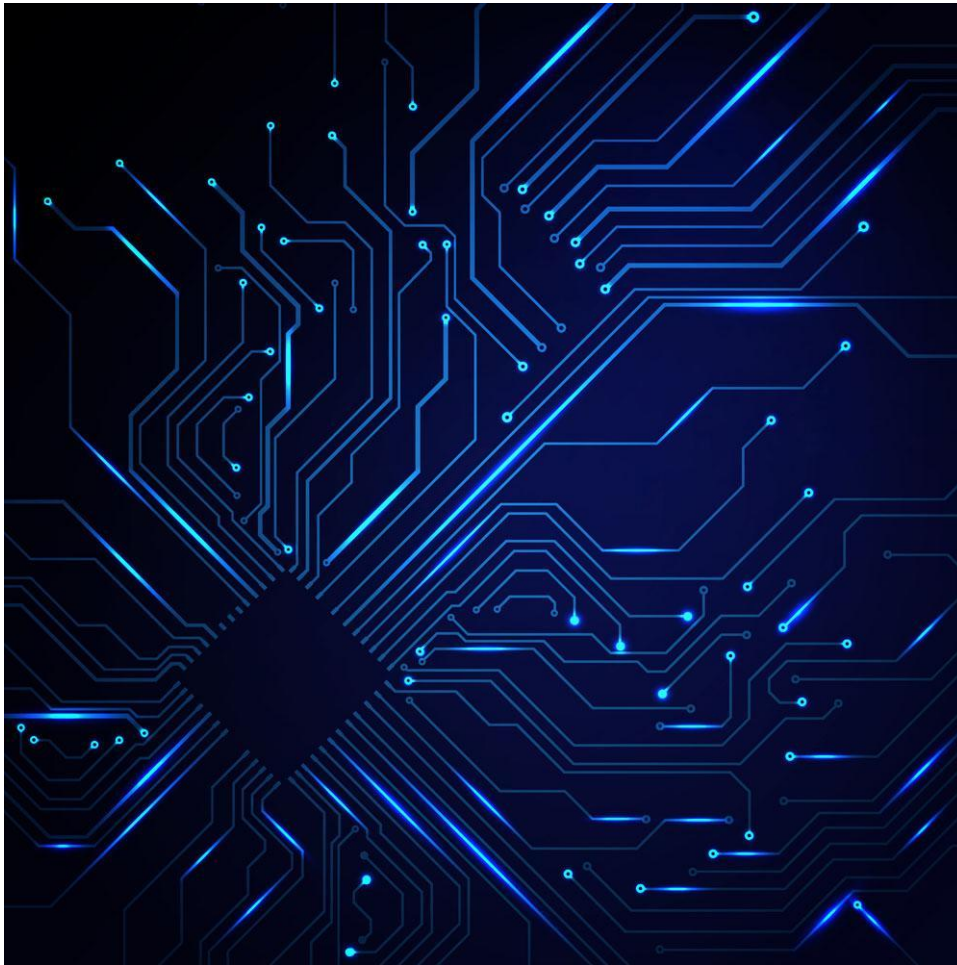




注意

1. 繳交時一律轉 PDF 檔
2. 繳交期限為
隔週三上午九點
3. 一人繳交一份
4. 檔名：學號_HW?.pdf
檔名請按照作業檔名格式進行填寫
未依照格式不予批改

2026/05/11

實驗十

TDI Hardware Design

姓名：陳篁瑋 學號：01257024

班級：資工 3A

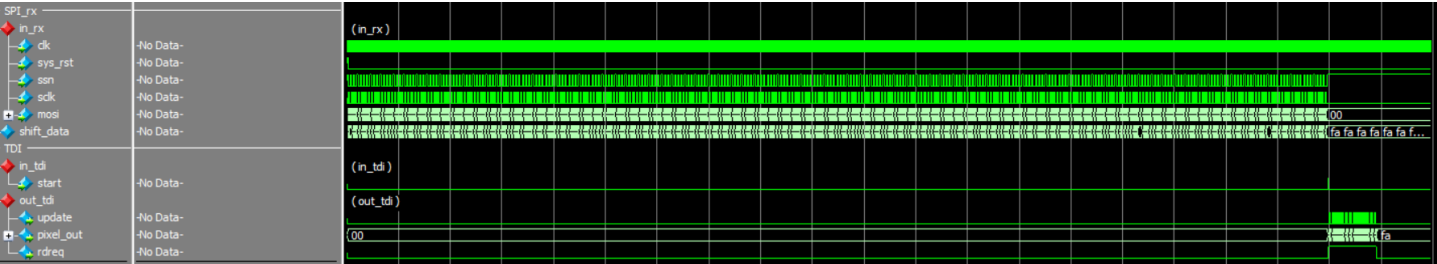
E-mail：henry246813579@gmail.com

TDI Hardware Design

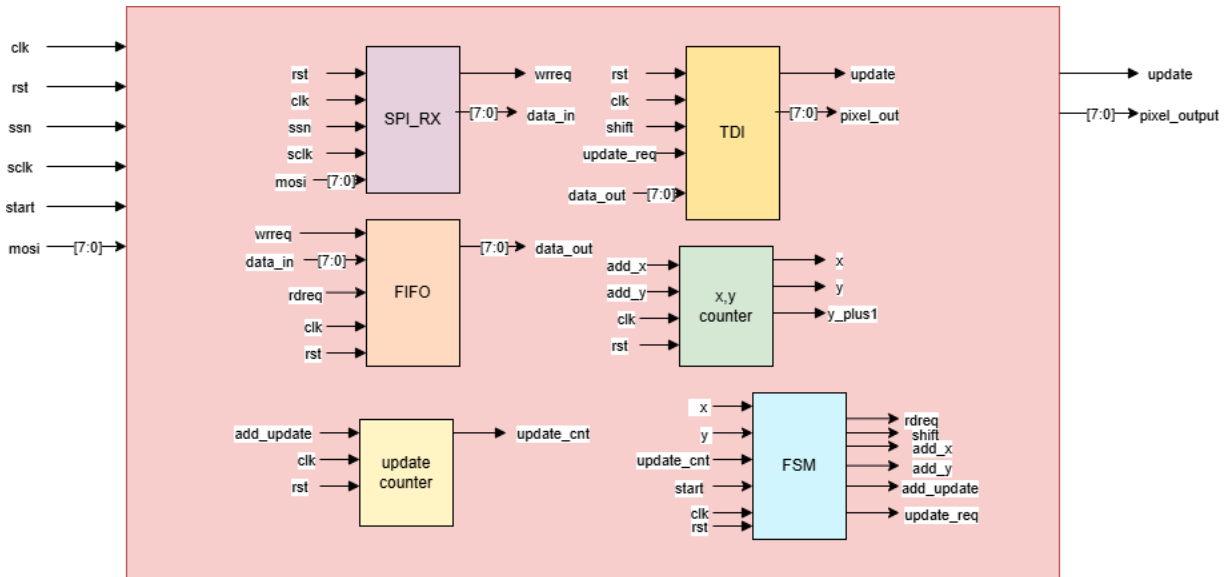
■ 實驗說明：

1. 請根據 TDI (Time Delay Integration) 演算法設計一個硬體架構。在此 LAB 中一張影像有 8×8 個像素，每個像素為 8 位元，testbench 一共 37 張影像。
2. TDI 輸入：
- | 訊號 | 說明 |
|-----------|--|
| sys_rst | 系統重置訊號，為 1 時重置系統。 |
| clk | 系統時脈訊號，頻率為 50MHz。 |
| sclk | 串列時鐘訊號，在正緣時讀取一個像素。 |
| ssn | 當 ssn 由 1 降至 0 表傳輸開始；由 0 升至 1 則表示傳輸結束。 |
| mosi[7:0] | 8 位元的像素資料輸入。 |
| start | TDI 開始運算訊號。 |
3. TDI 輸出：
- | 訊號 | 說明 |
|------------------|---------------------------------------|
| update | 每當 TDI 輸出一個有效像素時，發出一個時脈週期的 update 訊號。 |
| TDI_pixel_output | TDI 的像素輸出。 |
4. 若 ModelSim 模擬結果正確，Transcript 視窗將顯示 "Files are identical."；否則，將顯示 "Files differ."

■ 波型圖參考



■ 系統架構圖或方塊圖說明(請繪製自己設計的硬體架構圖並說明) 繪圖請善用 Draw.io



SPI_RX

接收 mosi[7:0] 的資料，產生 data_in[7:0] 像素資料，並同步拉高 wrreq 把資料存入 FIFO。

FIFO

接收來自 SPI_RX 的資料，並等待 FSM 發出 rdreq 時，將資料 data_out[7:0] 給 TDI。

TDI

接收 FIFO 的資料後，根據 shift 訊號執行累加運算

當完成 8 次累加後，將數值除以 8，從 pixel_out 輸出像素，並拉高 update。

x, y counter

FSM 發出 add_x 或 add_y 時，計數器會加一。

他會輸出 x, y, y_plus1 給 TDI 運算的座標，也會回傳給 FSM 作為狀態切換判斷。

update counter

記錄已經產出了多少個 pixel_output，決定 FSM 什麼時候要到 FINISH 這個狀態。。

FSM

讀取 start 訊號與座標狀態 x, y, update_cnt，產生所有控制訊號來控制所有硬體動作。

■ 系統架構程式碼與程式碼說明

截圖請善用 win+shift+S

TDI(top)

```
1 module TDI(
2     input logic clk,
3     input logic rst,
4     input logic ssn,
5     input logic sclk,
6     input logic [7:0] mosi,
7     input logic start,
8     output logic [7:0] pixel_output,
9     output logic update
10 );
11
12 //SPI_RX
13 logic wrreg;
14 logic rdreg;
15 logic [7:0] data_in;
16
17 spi_rx spi_rx_1(
18     .clk      (clk),
19     .rst      (~rst),
20     .ssn      (ssn),
21     .sclk     (sclk),
22     .mosi     (mosi),
23     .write_en (wrreg),
24     .data     (data_in)
25 );
26
27 //FIFO
28 logic [10:0] shift_reg [63:0];
29 logic [7:0] data_out;
30 logic [10:0] usedw;
31 logic almost_empty;
32 logic sclr;
33 logic full;
34 logic empty;
35
36 fifo fifo_1(
37     .clock      (clk),
38     .data       (data_in),
39     .rdreq      (rdreg),
40     .sclr       (rst),
41     .wrreq      (wrreg),
42     .almost_empty(almost_empty),
43     .empty      (empty),
44     .full       (full),
45     .q          (data_out),
46     .usedw      (usedw)
47 );
48
49 logic fifo_req;
50 logic trg_tdi;
51 integer i, j;
52 logic [31:0] update_cnt;
53 logic [2:0] x;
54 logic [2:0] y;
55 logic [2:0] y_plus1;
56 logic add_x;
57 logic add_y;
58 logic add_update;
59 logic shift;
60 //Counter
61 always_ff@(posedge clk)
62 begin
63     if(rst) begin
```

```

64         x         <= 0;
65         y         <= 0;
66         y_plus1   <= 1;
67         update_cnt <= 0;
68     end
69     if(add_x)
70         x <= x+1;
71     if(add_y) begin
72         y <= y+1;
73         y_plus1 <= y_plus1+1;
74     end
75     if(add_update)
76         update_cnt <= update_cnt + 1;
77     end
78
79     logic update_req;
80
81     always_ff@(posedge clk)
82     begin
83         if(rst)
84             update <= 0;
85         else
86             update <= update_req;
87         end
88
89     logic [10:0] deb_a;
90     logic [10:0] deb_b;
91
92     //TDI
93     always_ff@(posedge clk)
94     begin
95         if(rst)
96             for (i=0; i<64; i=i+1)
97                 shift_reg[i] <= 11'b0;
98     else if(shift) begin
99         if(y == 0) begin
100             pixel_output <= shift_reg[0] >> 3;
101         end
102         if(y_plus1 == 0)
103             shift_reg[8*y+7] <= {3'b0,data_out} + 0;
104         else
105             shift_reg[8*y+7] <= {3'b0,data_out} + shift_reg[8*y_plus1+x];
106         shift_reg[8*y+6] <= shift_reg[8*y+7];
107         shift_reg[8*y+5] <= shift_reg[8*y+6];
108         shift_reg[8*y+4] <= shift_reg[8*y+5];
109         shift_reg[8*y+3] <= shift_reg[8*y+4];
110         shift_reg[8*y+2] <= shift_reg[8*y+3];
111         shift_reg[8*y+1] <= shift_reg[8*y+2];
112         shift_reg[8*y+0] <= shift_reg[8*y+1];
113     end
114     end
115     assign deb_a = shift_reg[8*y+7];
116     assign deb_b = shift_reg[8*y_plus1+x];
117     //FSM
118     typedef enum{ INIT, READ_DATA, TDI, FINISH
119     } FSM_STATE;
120     FSM_STATE fsm_ns,fsm_ps;
121
122     always_ff@(posedge clk) begin
123         if(rst)
124             fsm_ps <= INIT;
125         else
126             fsm_ps <= fsm_ns;
127     end
128
129
130
131     always_comb
132     begin
133         fsm_ns = fsm_ps;
134         rdreq = 0;
135         shift = 0;
136         add_x = 0;
137         add_y = 0;
138         add_update = 0;
139         update_req = 0;
140         case(fsm_ps)
141             INIT:
142                 begin
143                     if(start)
144                         fsm_ns = READ_DATA;
145                 end
146             READ_DATA:
147                 begin
148                     rdreq = 1;
149                     fsm_ns = TDI;
150                 end
151             TDI:
152                 begin
153                     add_x = 1;
154                     shift = 1;
155
156                     if(update_cnt >= 8*36+1) begin
157                         fsm_ns = FINISH;
158                     end
159                     else if(update_cnt >= 1) begin
160                         if(y == 0) begin
161                             update_req = 1;
162                             add_update = 1;
163                         end
164                         if(x == 7)
165                             add_y = 1;
166                         fsm_ns = READ_DATA;
167                     end
168                     else begin
169                         if(x==7) begin
170                             add_y = 1;
171                             add_update = 1;
172                         end
173                         fsm_ns = READ_DATA;
174                     end
175                 end
176             FINISH:
177                 begin
178                     fsm_ns = INIT;
179                 end
180         endcase
181     end
182 endmodule
183
184

```

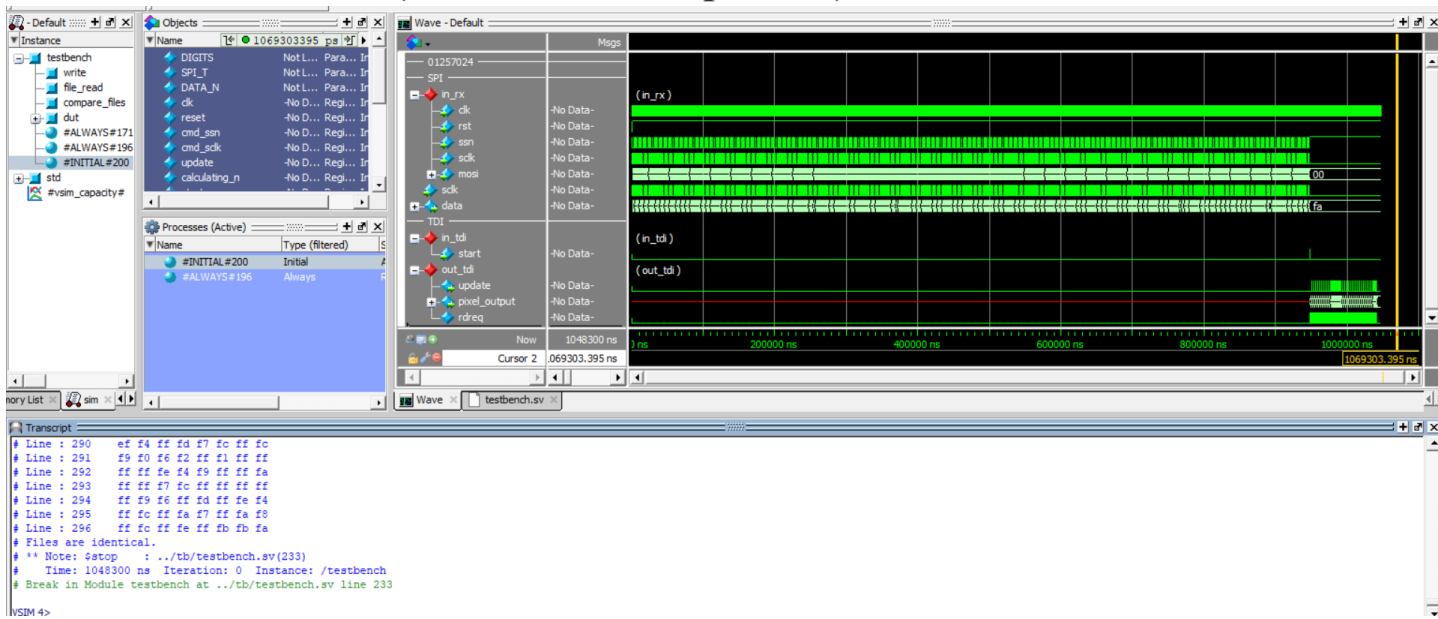
spi_rx

```

1 module spi_rx(
2     input logic clk,
3     input logic [7:0] mosi,
4     input logic ssn,
5     input logic sclk,
6     input logic rst,
7     output logic [7:0] data,
8     output logic write_en
9 );
10
11
12 //Edge Detector
13 logic sclk_s;
14 logic sclk_d;
15 logic ssn_s;
16 logic ssn_d;
17 logic ssn_negedge;
18 logic sclk_posedge;
19 logic sclk_negedge;
20
21 always_ff@(posedge clk)
22 begin
23     if(~rst)
24     begin
25         sclk_s      <= 1'b0;
26         sclk_d      <= 1'b0;
27         ssn_s       <= 1'b1;
28         ssn_d       <= 1'b1;
29     end
30     else
31     begin
32         {sclk_d, sclk_s} <= {sclk_s, sclk};
33         {ssn_d, ssn_s}  <= {ssn_s, ssn};
34     end
35 end
36
37 assign ssn_negedge = ~ssn_s & ssn_d;
38 assign sclk_posedge = sclk_s & ~sclk_d;
39 assign sclk_negedge = ~sclk_s & sclk_d;
40
41 //Counter
42 logic [31:0] receive_data_counter;
43 logic sclk_data_counter_rst;
44
45 always_ff@(posedge clk)
46 begin
47     if(~rst || ssn)
48         receive_data_counter <= 32'b0;
49     else if(sclk_posedge)
50         receive_data_counter <= receive_data_counter + 1'b1;
51 end
52
53 always_ff@(posedge clk)
54 begin
55     if(~rst)
56         data <= 8'b0;
57     else if(ssn_negedge)
58         data <= 32'b0;
59     else if(~ssn && sclk_posedge)
60         data <= mosi;
61 end
62
63
64 //FSM
65 typedef enum{INIT, START_SPI_RX, FINISH, RECEIVE_DATA } FSM_STATE;
66 FSM_STATE fsm_ns, fsm_ps;
67
68 always_ff@(posedge clk) begin
69     if(~rst)
70         fsm_ps      <= INIT;
71     else
72         fsm_ps      <= fsm_ns;
73 end
74
75
76 always_comb
77 begin
78     fsm_ns          = fsm_ps;
79     write_en        = 0;
80     case(fsm_ps)
81     INIT:
82     begin
83         fsm_ns = START_SPI_RX;
84     end
85     START_SPI_RX:
86     begin
87         if(ssn_negedge) begin
88             fsm_ns = RECEIVE_DATA;
89         end
90         else
91             fsm_ns = START_SPI_RX;
92     end
93     RECEIVE_DATA:
94     begin
95         if(sclk_negedge) begin
96             write_en = 1;
97             if(receive_data_counter==8)
98                 fsm_ns = FINISH;
99         end
100     end
101     FINISH:
102     begin
103         fsm_ns = INIT;
104     end
105 endcase
106 end
107
108
109 endmodule
110

```

■ 模擬結果與結果說明(需截到 Transcript 視窗)：



■ 結論與心得：

這次感覺很多細節都要自己想，甚至架構也是，跟之前看這架構寫的感覺很不一樣，一直會懷疑自己。